

```
④ oBall.draw() # tell the Ball to draw itself

# 11 - Update the window
pygame.display.update()

# 12 - Slow things down a bit
clock.tick(FRAMES_PER_SECOND)
```

Listing 6-2: The new main program that instantiates a `Ball` and makes calls to its methods

If you compare this new main program with the original code in [Listing 5-6](#), you'll see that it's much simpler and clearer. We use an `import` statement to bring in the `Ball` class code ❶. We create a `Ball` object, passing in the window that we created and the width and height of that window ❷, and we save the resulting `Ball` object in a variable named `oBall`.

The responsibility of moving the ball is now in the `Ball` class code, so here we only need to call the `update()` method of the `oBall` object ❸. Since the `Ball` object knows how big the window is, how big the image of the ball is, and the location and the speed of the ball, it can do all the calculations it needs to do to move the ball and bounce it off the walls.

The main code calls the `draw()` method of the `oBall` object ❹, but the actual drawing is done in the `oBall` object.

Creating Many Ball Objects

Now let's make a slight but important modification to the main program to create multiple `Ball` objects. This is one of the real powers of object orientation: to create three balls, we only have to instantiate three `Ball` objects from the `Ball` class. Here we'll use a basic approach and build a list of `Ball` objects. In each frame, we'll iterate through the list of `Ball` objects, tell each one to update its location, then iterate again to tell each one to draw itself. [Listing 6-3](#) shows a modified main program that creates and updates three `Ball` objects.